

Outbox 与一致性

用户看见的事实 / 商家依赖的事件 / 客服查得到的状态

gomall · 业务侧 deck

今日大纲

一致性是用户看得见的事实

五角色视角：一致性意味着什么

双写问题：用户看到了什么

Transactional Outbox：让”事件”也归 DB 管

Publisher：业务参数的业务解释

Saga 协同：补偿事件的业务故事

缓存一致性：用户多久看到改价

业务码与压测：用数字背书

可靠性补强：幂等卡死与 MQ 自愈

真实事故复盘：用户怎么感知到的

业务承诺与边界

一句话：一致性是“用户看见的事实对齐”

- 用户视角的一致：“我下单 = 订单列表能看到”、“我退款 = 库存能回来”、“我付款 = 订单不再卡 WaitPay”。
- 商家视角的一致：“客户下了单 = 我收到发货事件”，不能“我没收到事件客户却来问发货”。
- 运营视角的一致：**GMV 实时口径**（订单中心）和 **对账口径**（财务）差异不超过 5 分钟。
- 客服视角的一致：“用户问的每一条状态我都能在 **outbox** 表里查到”——事件溯源是话术的底层支撑。
- SRE 视角的一致：**outbox status** = pending / sent / dead 三态可监控，dead 必报警。

业务能容忍的不是“零延迟”，是“看得见的窗口”

业务事件	用户可接受窗口	不可接受的事实
下单 → 订单列表	$\leq 1\text{ s}$	用户点支付时列表里没这单 ”我付了钱订单还在 Wait-Pay”
支付 → 订单状态变更	$\leq 2\text{ s}$	
改价 → 商品详情	$\leq 1\text{ s}$	用户结算时还按旧价
退款 → 库存回到 available	$\leq 5\text{ min}$	售罄信息持续 10 分钟以上
商家发货 → 用户消息推送	$\leq 10\text{ s}$	客户先收到货才收到通知
业务承诺 (SLO): 所有”用户感知一致性”统一进 P1 档 (99.9%, p99 < 200 ms); ”对账一致性”进 P2 档 (99%, 5 min 闭合)。		

gomall 真正在意的三对一致性

一致性对	业务后果（用户看到什么）	gomall 的兜底
DB $\leftrightarrow$ MQ	商家不发货 / 搜索看不到 / 库存不回	Outbox (本 deck 主线)
DB $\leftrightarrow$ Cache	用户看到旧价 / 旧库存	延迟双删 + SETNX 回源锁
跨服务步骤	半成品订单、超卖、漏退款	协同式 Saga + 补偿事件
三对一致性都对应过真实事故，下文逐一复盘。		

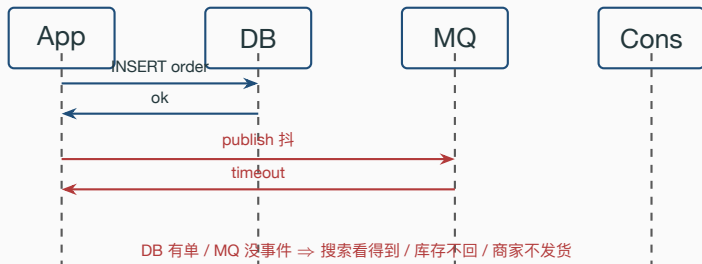
C 端用户：三类客诉

用户描述	真实根因	涉及一致性对
”我退款 24h 库存没回来”	outbox 事件 dead 未被发现	DB ↔ MQ
”我付了钱订单还显示有货”	ES 索引未及时增量更新	DB ↔ Cache(ES)
”订单一直卡在 Refunding”	Saga 补偿步骤其中一步 dead	跨服务
”我刚下的单订单列表没有”	outbox publish 抖动 > 1s	DB ↔ MQ
”商品页价格 5 块结算 6 块”	延迟双删窗口尚未关闭	DB ↔ Cache

对外话术：业务码沿用 gomall 真实编码——60002（重复请求处理中）、50001（库存不足/OSS 异常）、70001（限流）、70002（熔断）。

- **商家**：依赖 `order.created / order.paid / order.cancelled` 事件触发“配货 / 发货”工作流。一条事件丢 = 客户来问“为啥还没发货”。商家入驻 BossID 体系已落地，merchant 自助 API 路线图阶段。
- **运营**：GMV 实时口径来自订单中心实时累加，对账口径来自夜间 cron 扫表。两口径差异 $> 5 \text{ min}$ = 调价 / 大促数据失真。
- **客服**：每条客诉先在 `outbox_event` 表按 `aggregate_id=order_id` 查事件流，能直接看到这单 `OrderCreated` $\rightarrow$ `Paid` $\rightarrow$ `Cancelled` 的完整轨迹（含每次 `publish` 的 `attempts / last_error`）。
- **SRE**：监控 `outbox_pending_total > 10k / outbox_dead_total > 0 / outbox_publish_latency_p99 > 1s` 三个指标。

双写失败的客诉链路



用户感受:

- C 端：付了钱发现”商品页还显示有库存” → 错以为没付成功 → 重复下单。
- 商家：客户群里被催发货，自己后台没有这条单 → 找客服问。
- 客服：用户怒投诉时查不到任何事件，只能道歉 + 升级 SRE。

反方向也会炸：先 publish 再 commit

反向失败链路（更糟）：

1. App 先 publish "OrderCreated" 给 MQ → ok。
2. 接下来 DB commit **失败**（行锁冲突 / 唯一键冲突）。
3. 下游消费者已经按“幽灵订单”开始扣库存、推消息、走风控。
4. 用户视角：**我没收到下单成功的页面 / 没扣钱 / 却收到“订单已创建”短信。**
5. 客服视角：根本查不到这单订单号，但用户拿着短信来投诉。

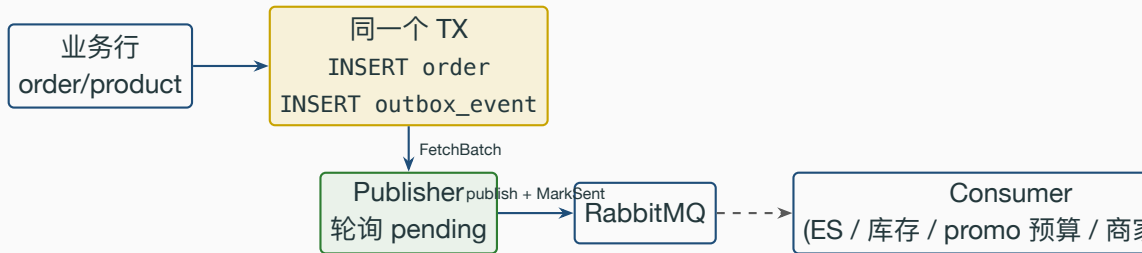
两个方向都不可接受 → 不能“两个 try/catch 套一下”→ 必须把事件“也当成 DB 行”写进事务。

为什么 gomall 不能用 2PC / XA

- **业务 RT 不接受**：协调者持锁全程，下单 p99 从 5ms 拉到 100ms+，违反 P0 SLO (< 500 ms 上限，但实际预算 5-10ms)。
- **异构资源不支持**：gomall 用 MySQL / Redis / RabbitMQ / ES 四个存储，只有 MySQL 支持 XA，其它三个都不行。
- **运维不可接受**：协调者宕机 = 所有参与者 prepared 阶段持锁；网络分区时数据被冻结，业务直接停摆。
- **业务可以接受最终一致**：1 秒内” 订单可见”、5 分钟内” 对账闭合” 是真实业务可接受的窗口。

业务结论：选 AP 路线，把不变量写在业务码 (pending/sent/dead) 里。

Outbox 的核心一句话



业务变更 + 事件写在同一个 DB 事务里——要么都成功，要么都失败。

”投递给 MQ”这件事被下沉到独立的轮询循环，业务请求不再等 MQ ack（用户感觉不到）。

outbox_event 表：客服查得到的事实流

```
25 type OutboxEvent struct {
26     dbmodel.Model
27     AggregateType string // "order" / "product" 客服按这个先过滤
28     AggregateID   uint  // 业务实体 id —— 客服按订单号 / 商品 id 直接查
29     EventType     string // "OrderCreated" / "OrderCancelled"
30     RoutingKey     string // "order.created" 投递到哪条 routing
31     Payload        string // JSON body —— 含一切下游需要的字段
32     Status         int    // 1 pending / 2 sent / 3 dead
33     Attempts       int    // 已尝试投递次数 (>0 = 抖动过)
34     NextRetryAt    time.Time // 指数退避后的下一次允许时间
35     LastError      string // 最后一次失败原因, 截断 500 字 (值班复盘用)
36 }
```

客服 SOP： 用户报”退款没回来”→ `SELECT * FROM outbox_event WHERE aggregate_type='order' AND aggregate_id=? ORDER BY id` → 看最后一行 Status: 1 = 还在排队请稍等 / 2 = 已送达需查下游 / 3 = 需升级 SRE。

CancelUnpaidOrder: 业务变更 + 事件同事务

```
31 err = baseDao.DB.Transaction(func(tx *gorm.DB) error {
32     ok, err := NewOrderDaoByDB(tx).CloseOrderWithCheck(orderNum)
33     if err != nil {
34         return err
35     }
36     if !ok {
37         return nil // 已经被关过，幂等 no-op
38     }
39     closed = true
40     return outbox.NewOutboxDaoByDB(tx).Insert(
41         "order", "OrderCancelled", "order.cancelled", order.ID,
42         events.OrderCancelled{
43             OrderID: order.ID, OrderNum: orderNum,
44             UserID: order.UserID, ProductID: order.ProductID,
45             Num: order.Num, Reason: "timeout",
46             PromoRuleID: order.PromoRuleID,
47             PromoDiscountCents: order.PromoDiscountCents,
48         },
49     )
50 })
```

业务读法：闭包出错自动回滚 → “关单 + 事件”一起可见或一起未发生；

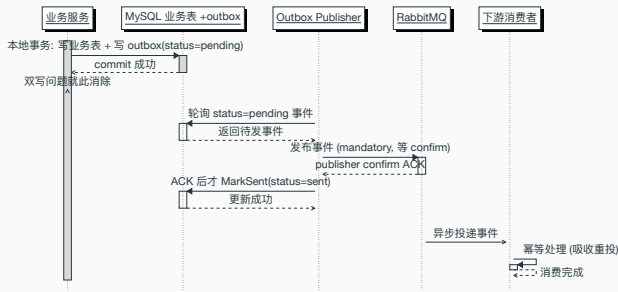
CloseOrderWithCheck 仅对 UnPaid 生效 = Saga 步骤的业务幂等键；客服重复点关单按钮也不会重复发事件。事件载荷自带 promo_rule_id/promo_discount_cents，满减预算退还由 promo.release 消费者（绑 order.cancelled/order.refunded）异步完成，消费侧不反查订单表。

Outbox 怎么消除两个“不对称失败”

- 失败方向 A (DB 成功 / MQ 失败): 事件**已经**写进 outbox 表, publisher 下一轮自动捞起来重发 → 客户最多等 1 秒看到下单成功。
- 失败方向 B (DB 失败 / MQ 已发): 事件**从未**进 outbox 表 (事务回滚), 下游永远不会收到“幽灵订单”。
- 业务承诺: 从用户点“提交订单”到订单出现在订单列表 ≤ 1 秒 (PollInterval 上限)。
- 商家承诺: 从用户支付成功到商家后台收到 `order.paid` 事件 ≤ 1 秒 + 消费者处理时间。

Outbox 把“两个存储双写”降级为“一个 DB 事务 + 一个异步轮询”。

Transactional Outbox 可靠投递时序



先落库再异步投递、ACK 后才标记已发：DB 成功的事件最终一定送达；消费侧幂等吸收重投。

默认参数：每个数值背后是业务约束

参数	默认值	业务依据
PollInterval	1 s	用户从下单到” 订单列表可见” 的可接受窗口；搜索/对账也按这个量级
BatchSize	100	100 行单次能在 50ms 内 commit 完，不让 publisher 自己变慢查询
MaxAttempts	10	$1+2+4+8+16+32+64+128+256+300 \approx 13.5 \text{ min}$ 累计窗口，盖一次中等故障

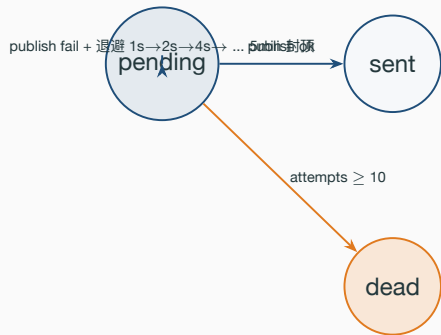
业务可解释口径：

- PollInterval=1s = 用户/商家/搜索的” 用户感知一致性” 上限。
- BatchSize=100 = 不让 outbox 自己拖慢业务写入。
- MaxAttempts=10 = 10 次失败后人工兜底，避免无限重试打死下游。

publisher.loop / drainBatch: 业务流水线

```
49 func (p *Publisher) loop(ctx context.Context) {
50     ticker := time.NewTicker(p.cfg.PollInterval); defer ticker.Stop()
51     for {
52         select {
53             case <-ctx.Done(): return
54             case <-ticker.C: p.drainBatch(ctx)
55         }
56     }
57 }
58 func (p *Publisher) drainBatch(ctx context.Context) {
59     if rabbitmq.GlobalRabbitMQ == nil { return } // RMQ 静默降级: 未起则跳过
60     rows, err := NewOutboxDao(ctx).FetchBatch(p.cfg.BatchSize)
61     if err != nil { util.LogrusObj.Errorln(err); return }
62     for _, ev := range rows {
63         if err := rabbitmq.PublishDomainEvent(ctx, ev.RoutingKey, []byte(ev.Payload)); err != nil {
64             NewOutboxDao(ctx).MarkFailed(ev.ID, ev.Attempts, p.cfg.MaxAttempts, err.Error())
65             continue
66         }
67         NewOutboxDao(ctx).MarkSent(ev.ID)
68     }
69 }
```

业务码状态机： pending / sent / dead



- **pending**: 等下一轮 publisher 取（客服话术：“正在投递，请稍等”）。
- **sent**: 事件已送达 broker，下游处理中（客服话术：“已转交对应服务”）。
- **dead**: 人工介入分支——看 LastError，找下游问题；**不会自愈**（客服话术：“已升级技术团队，请留联系方式”）。

MarkFailed: 指数退避 + 死信落地

```
62 func (d *OutboxDao) MarkFailed(id uint, attempts, maxAttempts int, errMsg string) error {
63     now := time.Now()
64     updates := map[string]any{
65         "attempts": attempts + 1,
66         "last_error": truncate(errMsg, 500),
67         "updated_at": now,
68     }
69     if attempts+1 >= maxAttempts {
70         updates["status"] = OutboxStatusDead
71     } else {
72         backoff := time.Duration(1<<attempts) * time.Second
73         if backoff > 5*time.Minute { backoff = 5 * time.Minute }
74         updates["next_retry_at"] = now.Add(backoff)
75     }
76     return d.DB.Model(&OutboxEvent{}).Where("id = ?", id).Updates(updates).Error
77 }
```

业务读法：1 << attempts 退避 1s/2s/.../256s，被 5 min 封顶；next_retry_at 写入让 FetchBatch 下一轮自动过滤掉；第 10 次失败进 dead 触发报警。

死信处理：让”失败可见”而不是”自动忽略”

- dead 状态不会自愈——设计选择：让人介入比代码盲目重试更安全。
- 监控指标（业务承诺）：
 - `outbox_pending_total > 10k`：堆积报警，影响用户感知。
 - `outbox_dead_total > 0`：累计死信，每出现就报警。
 - `outbox_publish_latency_p99 > 1s`：违反 1 秒承诺。
- 客服 **SOP**：用户报投诉 → 查 `outbox` 表 → `status=3 dead` → 升 SRE → SRE 确认下游修复 → `UPDATE outbox_event SET status=1, attempts=0, next_retry_at=now() WHERE id=?` → publisher 下一轮自动重发。
- 业务承诺：**outbox sent rate $\geq$ 99.99%**（每万条事件最多 1 条进 dead）。

死信表是客服 + **SRE** 的事实库，不是”代码兜底箱”。

协同式 Saga: 5 步链路 + 任一失败补偿

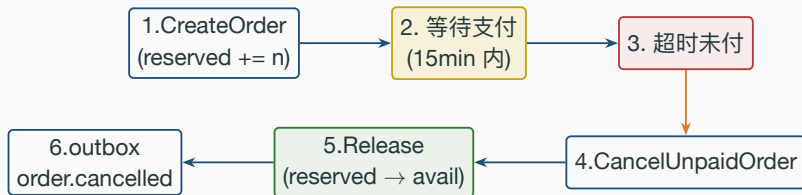
模式	实现	业务取舍
编排 (Orchestration)	中心 Coordinator 推	步骤 ≥ 5 、需要可视化看板
协同 (Choreography)	事件驱动、无中心	gomall 选用 ——步骤少、跨团队少



每一步消费上一步事件、产出下一步事件，无总指挥。

对 gomall 的 5 步链路是最省心的选择——加新下游 = 新加一个消费者，不改主链路。

Saga 失败: CancelUnpaidOrder 就是一次补偿



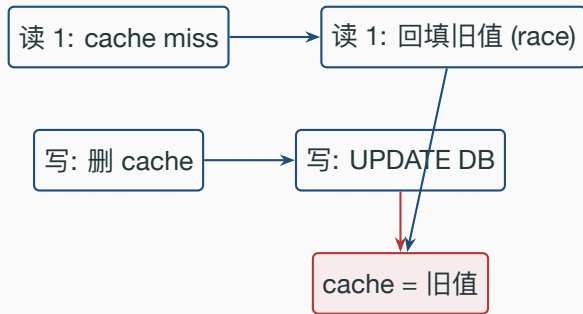
业务承诺:

- **Saga 补偿 ≤ 5 min**: 超时关单 + 库存回填全链路 5 分钟内闭合。
- **库存不变量**: $available + reserved + sold = total$ 始终成立。
- **先 DB tx 后 cache**: DB 关单 + outbox 一起 commit 后才释放 Redis 占用, 失败可重放。
- **满减预算同样走补偿事件**: `order.cancelled / order.refunded` → `promo.release` 消费者退预算, at-least-once 重复投递由 `promo_release` 台账 (OrderID 唯一索引, INSERT 冲突即“已释放过”) 幂等吸收。

补偿失败的容错：用户感受是什么

- Saga 补偿不是银弹：补偿本身也会失败（cache 拒连 / 网络抖）。
- **gomall 的容错思路：**
 1. 先保证 DB commit ——业务真相先落地（订单确实关了）。
 2. Cache 释放失败只打 error 日志，不回滚 DB ——状态机靠下次扫描自愈。
 3. 真出现长期不一致时，由 cron + outbox 死信表兜底。
- **用户感受：**库存可能”短暂少一点”（业务可接受），但绝不”卖超”（业务不可接受）。
- **客服话术：**”您的订单已成功取消，库存回填中，最迟 5 分钟可见。”

缓存出错的用户感知



业务出错形态：

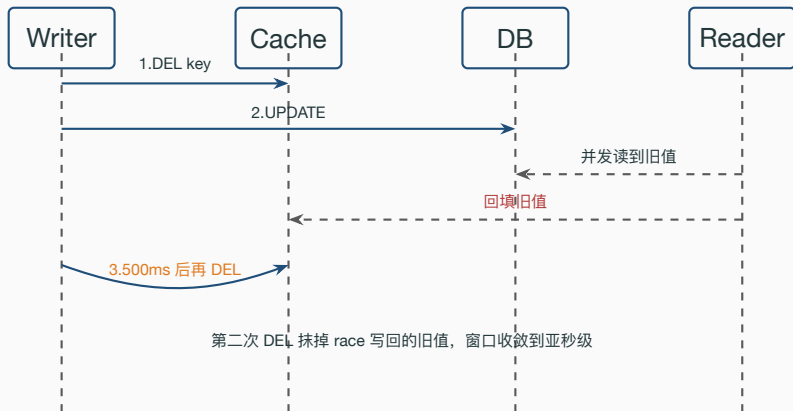
- C 端：商品改价后短时间还看到旧价 → 用户截图投诉” 虚假宣传”。
- 商家：调完价想” 立刻看到生效” → 自己刷商品页还是旧的 → 怀疑改价没成功 → 重复操作。
- 运营：促销开始的瞬间，部分用户看到的还是原价 → 大促转化率统计失真。

改价生效窗口：三方期望对齐

角色	期望窗口	gomall 实际	满足
C 端用户	改价后 ≤ 5 分钟 看到新价	$\leq 1\text{ s}$ (延迟双删)	✓ 远超期望
商家	改价后立刻自己 能看到生效	写后立即 DEL, 自己 读必 miss 回源	✓
运营平台	全网用户在大促 开始秒级同步	$\leq 1\text{ s} + \text{ES outbox 增}$ 量索引	✓

业务承诺：缓存延迟双删 $\leq 1\text{ s}$ 内全部生效（实测 500 ms 抹掉所有 race）。

延迟双删时序图

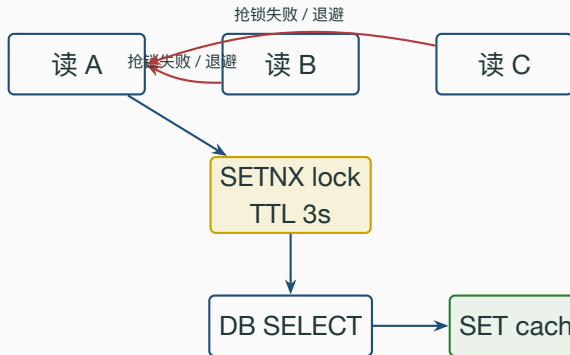


双删 + 击穿保护：源码

```
102 // TryProductLock SETNX 抢回源锁，避免缓存击穿时多个请求同时回源
103 func TryProductLock(ctx context.Context, id uint) (bool, error) {
104     return RedisClient.SetNX(ctx, ProductLockKey(id), "1", ProductLockTTL).Result()
105 }
106
107 func UnlockProduct(ctx context.Context, id uint) {
108     RedisClient.Del(ctx, ProductLockKey(id))
109 }
110
111 // DoubleDeleteAsync 延迟双删：写库后已经做了第一次删除，这里在 interval 后再删一次，
112 // 用于覆盖"读旧值的并发请求把旧值塞回缓存"的窗口。
113 func DoubleDeleteAsync(id uint, interval time.Duration) {
114     if interval <= 0 { interval = ProductDelayInterval } // 默认 500ms
115     go func() {
116         time.Sleep(interval)
117         _ = RedisClient.Del(context.Background(), ProductDetailKey(id)).Err()
118     }()
119 }
```

业务读法：SetNX 单进程拿回源锁；ProductLockTTL=3s 防死锁；goroutine 不阻塞写接口（商家点保存即时返回）；context.Background() 因为商家请求已返回但 500 ms 后还要补一刀。

击穿保护：SETNX 单飞回源



业务收益：首页 banner 商品 cache 失

效瞬间不会让 1000 个请求一起打到 DB；落库次数从 N 降到 $1 + \text{其余复用 cache}$ 。

用户感受：首页商品永远秒开，永远不会“突然变慢 5 秒”。

业务码全表对照

业务码	含义	与一致性的关联
60002	重复请求处理中	幂等 Lua 状态机, 下游消费 outbox 必须
50001	OSS / 库存等下游异常	缓存击穿 / 回源失败 / Saga 补偿入口
70001	限流	客户端重试触发 = 重复事件 = 必须幂等
70002	熔断	下游 dead 时上游主动断
30001/30002/30005	鉴权三类失败	客服优先排除身份问题
20001	boss token	商家自助 API 鉴权 (路线图)

所有业务码全表见 pkg/e/code.go, 客服话术配套见 README 业务全景表。

压测里量出来的两组真实数字

压测场景

单元测试 TestCoupon_AtomicClaimNoOversell: 500 并发抢 100 张券

压测 idempotency_replay.js: 同 Key 累计 15s 请求

对应入库订单数 (SELECT COUNT(*))

回放 p95 / 吞吐

2.33 ms / **50,319**

基线 / ping

3.51 ms / 64,250

”零超发” + ”75 万次只入 1 单” 是同一套设计的两个表现——**at-least-once** 投递遇上业务

码幂等键 (券号 / Idempotency-Key), 最终对账数永远只算一次。

Outbox 解决”丢失”，幂等解决”重复”



投递语义：**at-least-once** ——至少一次。

at-least-once + 业务幂等键 = 业务侧 exactly-once。

困局一：幂等记录卡在 processing 永不可重试

业务困局：下单 / 支付带 Idempotency-Key，中间件先置 processing，业务跑完再推进到 done。若这步提交失败（Redis 抖动 / 请求 ctx 已取消），旧实现只打日志——记录永久停在 processing，同 Key 再来一律返回 60002，直到 TTL 过期才解锁。此时业务副作用已落库、2xx 已返回，既不能忽略也不能盲目重放，用户视角是“付款反复被挡、单子既没成也不能重下”。

```
122 func commitIdempotencyResult(key, result string) {
123     for i := 0; i < idempotencyCommitRetries; i++ {
124         ctx, cancel := context.WithTimeout(context.Background(), idempotencyCleanupTimeout)
125         err := cache.CommitIdempotencyResult(ctx, key, result)
126         cancel()
127         if err == nil { return }
128         log.LogrusObj.Errorln("idempotency commit error:", err)
129     }
130     releaseIdempotencyLock(key) // 回退 init，优先保证客户端可重试
131 }
```

修复：提交失败改为后台独立 context 带超时重试若干次；仍失败则把记录回退到 init，让客户端用同一 Key 安全重试，而非滞留 processing。

困局二：毒丸消息无限 requeue 打爆队列

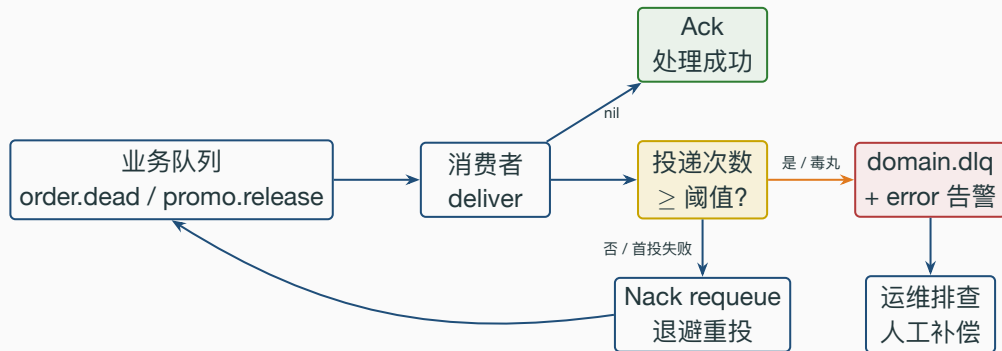
业务困局：消费失败时 `Nack(requeue=true)` 会把消息退回队首立刻重投。若是不可恢复的毒丸消息（body 解析失败 / 未知 routing key），它会被无限回灌：

- 单条坏消息把消费者 100% CPU 占住，正常消息全部饿死 → 满减预算不退、延迟关单停摆。
- 旧实现靠”判断 error 类型”决定是否丢弃，脆弱：业务把所有错误都判成可重试，兜底就失效。

修复：改成按投递次数判定。优先读 broker 维护的 `x-death header`（跨重启可靠的死信计数），退化用 `Redelivered`；次数达阈值就显式投到独立 DLX/DLQ 并打 error 告警，再 Ack 原消息——即便错误分类失灵也不会无界回灌。

- 用独立 `domain.dlx / domain.dlq`，不给存量业务队列加 `x-dead-letter-exchange` 参数，避免 broker 406 `PRECONDITION_FAILED`。
- 投 DLQ 本身失败时**不 Ack**，退回 `Nack(requeue)` 靠下一轮兜底，守住 `at-least-once` 不丢。

毒丸消息超阈值进 DLQ 的流转



两条进 DLQ 的路径：消费侧错误分类判为不可恢复（poison）或投递次数超阈值兜底。两路统一进独立 DLQ，不混进业务队列。

order.dead.queue 未配 DLX，首次失败允许重投一次，再失败（Redelivered）即转 DLQ；
promo.release 则 poison 或超限即转。

困局三：消费者断连后静默停摆 + 生产 fail-fast

业务困局：amqp091-go 不做自动恢复。broker 重启 / 网络抖动关掉投递通道后，旧消费者的 for range msgs 直接退出 goroutine —— **静默死亡**，无报错无重连，延迟关单 / 满减退预算 / 异步下单全部停摆。另一面：旧 InitRabbitMQ 连不上时内部 **panic / 吞错**，生产行为不可控。

```
26 func superviseConsumer(cfg consumerConfig) {
27     go func() {
28         for {
29             ch, msgs, err := subscribe(cfg)
30             if err != nil { time.Sleep(consumerRetryDelay); continue }
31             for d := range msgs { cfg.deliver(d) }
32             _ = ch.Close() // 通道被关：释放句柄
33             time.Sleep(consumerRetryDelay) // 退避后回到顶部重订阅
34         }
35     }()
36 }
```

修复：消费者跑在自愈 supervisor 里，通道关闭即 Close 旧句柄、退避后经 ensureConnection 重连重订阅；断连翻转 healthy 标记供探活，启动按 RequireOnStartup 决定 fail-fast 还是降级。

事故一：未支付订单取消导致虚高库存

用户感知 (commit 5c9ae27 前)：商品页” 库存 999”→ 真去下单” 库存不足”→ 一头雾水。

根因链：

1. 旧版 CancelUnpaidOrder 把 product.num 加回 DB ——但未支付订单从未真正扣减过 DB 库存（只占了 Redis reserved）。
2. 第一次取消：DB.num += n；Redis.reserved -= n ——库存虚高 n。
3. 累计 100 个超时订单 = 库存虚高 100 倍。

修复 (PR #58 引入预扣 + Saga)：

- DB.num 只在**支付成功**时递减，未支付订单只动 Redis reserved。
- CancelUnpaidOrder **不再**写 product.num，只释放 Redis reserved。
- 库存不变量：available + reserved + sold = total 始终成立。

事故二：搜索索引腐烂 12 小时无人知

用户感知 (commit a43cf61 前)：商品改价后，搜索结果一直显示旧价；商详情页正常。用户点进去发现价格不一样 → 投诉” 虚假宣传”。

根因链：

1. 旧链路：商品 UPDATE 时同步调用 ES 重建索引。
2. ES 集群偶发拒连 → 同步调用失败 → 业务 rollback → 商家投诉” 我改不了价”。
3. 业务为了不影响下单，把 ES 写入改成忽略错误。
4. ES 失败被吞，索引与 DB 漂移 12 小时无人知。

修复 (PR #59 切到 outbox + 消费者)：

- 商品 UPDATE 同事务只写 **outbox_event**，不直接写 ES。
- ES 消费者从 RMQ 拉事件，失败有重试 + 死信。
- 死信触发 alarm，值班 5 分钟内看到 (SRE SOP)。

事故三：InitLog 顺序错让全站 503

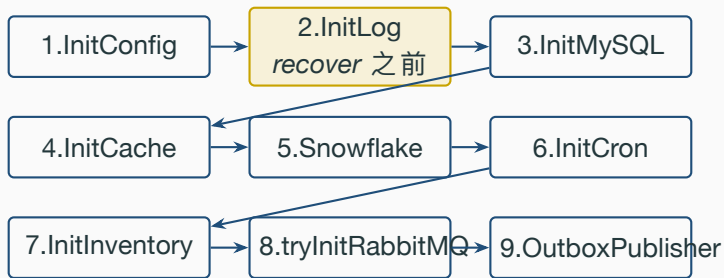
用户感知：RabbitMQ 服务下线维护时，gomall 进程**整体退出**，HTTP 接口全 503
→ **GMV 归零**。

根因链 (branch fix/init-log-order-before-rmq)：

1. tryInitRabbitMQ() 的 recover 块里调用 LogrusObj.Warnf。
2. 当时 util.InitLog() 反而在它之后。
3. LogrusObj == nil → recover 内再次 panic → 进程退出。
4. RMQ 应该是可选依赖 (延迟关单可降级)，实际把整个进程拖死。

业务影响：服务起不来 = 全站瘫痪 = GMV 归零 = P0 事故。

修复后的启动顺序（业务因果）



关键约束：**InitLog** 必须在 **InitMySQL / RMQ** 之前，因为后续所有 **tryInit*** 的 **recover** 都依赖 **LogrusObj**。

两条可复用的业务侧教训

1. **初始化顺序**：所有依赖全局对象的代码，必须在它初始化之后才能调用。
 - InitLog / InitTracer / InitMetrics 放最前。
 - 业务承诺：所有外部依赖独立 tryInit* 包装 + recover，主链路（下单/支付/列表/详情）永远在线。
2. **recover 内禁止再 panic**：
 - recover 是最后兜底，里面再崩进程彻底失控。
 - 写法：recover 内只 `fmt.Fprintln(os.Stderr, ...)` 或写本地文件，不依赖任何复杂组件。

outbox publisher 起不来不算事故——启动顺序错了让全站 503，才是事故。

三档一致性窗口：写在 SLO 里

档位	窗口上限	业务场景	gomall 落点
强一致	100 ms 内	同一事务可见性、行锁内的预扣	库存预扣 (Redis Lua)
近实时一致	1 s 内	用户感知”操作完成 = 立刻可见”	Outbox publish + ES 索引
最终一致	5 min 内	跨服务对账、数仓同步	死信复盘、cron 巡检

对外业务承诺：

- 用户下单后 $\leq 1\text{ s}$ 在订单列表看到（近实时档）。
- 商品改价后 $\leq 1\text{ s}$ 商品详情显示新价（双删窗口）。
- 库存与订单的对账差异 $\leq 5\text{ min}$ 闭合（夜间 cron）。
- Outbox sent rate $\geq 99.99\%$ （每万条事件最多 1 条进 dead）。
- Saga 补偿 $\leq 5\text{ min}$ 全链路闭合。

业务边界：本一致性方案不做什么

诚实列出（参考 README § 业务边界）：

- 不用 **2PC / XA**：协调者持锁拖死 p99，业务 RT 不可接受；MySQL 之外的存储都不支持 XA。
- 不用 **TCC**：每个接口要写 Try / Confirm / Cancel 三份，业务侵入太重；gomall 用 outbox 异步事件做等价语义。
- 不用 **binlog CDC**（Canal / Debezium）：表数量 ≈ 12 、消费者全 Go、单 publisher 没瓶颈，引入 CDC ROI 不正。
- 不做事件溯源全量重放：outbox 表只追加（无 Delete 路径），为未来演进留门，但目前不维护快照 / 投影。
- 不做跨地域多活同步：单机房部署，多活留给规模到达后再上。
- 不做 **RocketMQ** 事务消息：gomall 用 RabbitMQ，事务消息是 RocketMQ 专属能力。

业务侧总结：一致性约束一张表

一致性对	模式	业务码 / 表	gomall 现状
DB ↔ MQ	Transactional Outbox	outbox_event.status	pending/sent/dead
DB ↔ Cache	Cache Aside + 延迟双删	product:detail:{id}	500 ms 双删
缓存击穿	SETNX 单飞回源	product:lock:{id}	TTL 3 s
跨服务 Saga	协同式 (事件驱动)	见 outbox 路由 key	5 步链路
启动依赖	顺序约束	N/A	InitLog 第 2 位
五对一致性各有客诉链路、各有兜底、各有业务承诺；缺一不可。			

面试 Q&A (1/3)

Q1: 为什么 Outbox 比"DB commit 后再 publish" 安全?

A: commit 与 publish 不在一个事务, commit 成功 publish 失败就丢事件 → 商家收不到发货事件、库存不回。Outbox 把事件写进 DB, 由轮询消化, 事件丢失等价于 DB 数据丢失。

Q2: outbox 表会不会堆积?

A: 会。监控 `status=pending AND next_retry_at <= now()` 的行数即可。横向扩展用 `SELECT FOR UPDATE SKIP LOCKED`; 当前 gomall 单实例 `BatchSize=100/1s` 还远没到瓶颈。

Q3: 延迟双删的"延迟"该取多大?

A: 取大于最长读 race 窗口。gomall 取 500ms: DB 写常态 5-20ms, 留 20× 余量盖 GC / 网络抖动; TTL 10min 兜底。

Q4: 协同式 vs 编排式 Saga, 怎么选?

A: 步骤 ≤ 5 、跨团队少 → 协同 (gomall 选)。步骤多、需可视化看板、跨团队多 → 编排, 但要承担"中心节点"运维成本。

面试 Q&A (2/3)

Q5: 为什么 cache 释放失败不回滚 DB?

A: DB 是业务真相, cache 是性能侧。Cache 失败让库存”短暂少一点”, 最坏拒单不超卖; 回滚 DB 反而让”已关单”被复活, 业务码乱套, 客服更难解释。

Q6: 客服来问”用户的退款 24h 没回来”怎么查?

A: `SELECT status, attempts, last_error FROM outbox_event WHERE aggregate_type='order' AND aggregate_id=? ORDER BY id`。status=1 排队中 / 2 已发下游 / 3 dead 需升 SRE。

Q7: 2PC / XA 为什么不用?

A: 协调者单点 + 同步阻塞 + 异构资源不支持 (Redis/RMQ/ES 都没 XA)。业务 RT 不可接受 (下单 p99 从 5ms 拉到 100ms+, 违反 SLO)。

Q8: 业务承诺的”1 秒可见”怎么保证?

A: `PollInterval=1s + BatchSize=100` + 单实例 $< 100 \text{ ev/s}$ 峰值 $\rightarrow$ 平均延迟 $< 500 \text{ ms}$, 最坏 1s。监控 `outbox_publish_latency_p99 > 1s` 报警。

面试 Q&A (3/3)

Q9: MaxAttempts=10 怎么算出来的？退避 5min 封顶又是为什么？

A: $1+2+4+8+16+32+64+128+256+300=811s \approx 13.5min$ 累计窗口，覆盖一次”中等故障”够用；封顶 5min 防止退避无限拉长导致积压堆 outbox 表。

Q10: 进 dead 的事件谁来处理？

A: 触发报警 → 值班 SRE 看 last_error → 联系下游确认修复 → SQL 改回 status=1, attempts=0, next_retry_at=now() → publisher 自动重发。**绝不自动改回 pending。**

Q11: 为什么 drainBatch 第一行要判 rabbitmq.GlobalRabbitMQ == nil？

A: 静默降级。RMQ 未起时 publisher 不发事件、不动 DB；主链路（下单 / 支付）继续在线，事件堆 outbox 等 RMQ 恢复后自动消化。

Q12: 业务侧怎么承诺”最终一致”的窗口？

A: 分三档：库存预扣 100ms、订单/搜索可见 1s、跨服务对账 5min。对外 SLO 写明上限，超过触发 P2 报警；客服话术按这三档给用户解释。

代码位置一览

outbox_event 表

Outbox DAO (Insert / FetchBatch / MarkSent / MarkFailed)

Publisher (Start / loop / drainBatch)

初始化 publisher

OrderCreate (业务 + outbox 同事务)

CancelUnpaidOrder (Saga 补偿)

promo.release 消费者 + 释放台账

缓存双删 / SETNX 回源锁

ProductUpdate 调用双删

启动顺序修复点

业务码全表

压测数据源

配套: deck 04-cart-to-order / deck 07-inventory / deck 09-order-lifecycle /
docs/blog/04-outbox-saga.md